SRU	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: Namoju pranitha Reg. No:2503A52461
	Department: Computer Science and AISemester: 3
	AI Assisted Coding-Assignment:12 Date: 22-9-2026

1.- AI-Assisted Bubble Sort Analysis

Question:

numbers = [64, 34, 25, 12, 22, 11, 90]

TODO:

Implement Bubble Sort in ascending order.

Count comparisons and swaps.

Display the sorted list.

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

prompt:

"Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet"

Code:

```python
numbers = [5, 1, 4, 2, 8]

for pass_number in range(len(numbers) - 1):
    swapped = False

    # Values at the end are already sorted, so skip them.
    for index in range(len(numbers) - 1 - pass_number):
        print(f"Compare {numbers[index]} and {numbers[index + 1]}")

        if numbers[index] > numbers[index + 1]:
            numbers[index], numbers[index + 1] = (
                numbers[index + 1], numbers[index]
            )
            swapped = True
            print(f"Swap -> {numbers}")

    print(f"After pass {pass_number + 1}: {numbers}")

    # No swaps means the list is already sorted.
    if not swapped:
        break
```

output:

```
Compare 5 and 1
Swap -> [1, 5, 4, 2, 8]
Compare 5 and 4
Swap -> [1, 4, 5, 2, 8]
Compare 5 and 2
Swap -> [1, 4, 2, 5, 8]
Compare 5 and 8
After pass 1: [1, 4, 2, 5, 8]
Compare 1 and 4
Compare 4 and 2
Swap -> [1, 2, 4, 5, 8]
Compare 4 and 5
After pass 2: [1, 2, 4, 5, 8]
Compare 1 and 2
Compare 2 and 4
After pass 3: [1, 2, 4, 5, 8]
```

Observation:

The program sorts the given numbers in ascending order using Bubble Sort. It compares adjacent numbers and swaps them when needed. After each pass, the largest number moves to the end. The swapped variable stops the program when no swaps are needed. This helps reduce unnecessary comparisons and makes the program more efficient.

2. – Linear Search vs Binary Search

Question:

Problem / Starter Code:

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

target = 45

TODO:

Implement Linear Search.

Implement Binary Search.

Compare the number of comparisons.

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

Prompt:

Act as an expert Python programmer. Implement Linear Search and Binary Search for the given sorted list and count the number of comparisons made by each algorithm. Test both algorithms with the first, middle, last, and absent elements. Explain why Binary Search requires sorted data, compare the results, mention suitable use cases for each search method, and suggest one valid optimization. give a code

Code:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]


def linear_search(values, target):
    comparisons = 0

    for index, value in enumerate(values):
        comparisons += 1

        if value == target:
            return index, comparisons

    return -1, comparisons


def binary_search(values, target):
    # Binary Search requires sorted data because it uses the
    # middle value to discard half of the remaining list.
    left = 0
    right = len(values) - 1
    comparisons = 0

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1

        if values[middle] == target:
            return middle, comparisons
        elif values[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons


test_values = {
    "First": numbers[0],
```

```
    "Middle": numbers[len(numbers) // 2],
    "Last": numbers[-1],
    "Absent": 40
}

for name, target in test_values.items():
    linear_index, linear_count = linear_search(numbers, target)
    binary_index, binary_count = binary_search(numbers, target)

    print(f"{name} element ({target}):")
    print(f"  Linear Search: index={linear_index}, comparisons={linear_count}")
    print(f"  Binary Search: index={binary_index}, comparisons={binary_count}")
```

output:

```
First element (5):
  Linear Search: index=0, comparisons=1
  Binary Search: index=0, comparisons=3
Middle element (31):
  Linear Search: index=4, comparisons=5
  Binary Search: index=4, comparisons=1
Last element (89):
  Linear Search: index=8, comparisons=9
  Binary Search: index=8, comparisons=4
Absent element (40):
  Linear Search: index=-1, comparisons=9
  Binary Search: index=-1, comparisons=3
```

Observation:

The program implements Linear Search and Binary Search successfully.
Linear Search checks each element one by one.
Binary Search checks the middle element and reduces the search range.
Binary Search works only when the list is sorted.
The number of comparisons is counted for both methods.
Binary Search uses fewer comparisons in most test cases.
Both methods were tested with first, middle, last, and absent elements.
Thus, Binary Search is efficient for large sorted lists.

3.- Compare Sorting Algorithms

Question:

Problem / Starter Code:

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.

Prompt:

Implement Bubble, Selection, and Insertion Sort. Count comparisons/swaps, compare performance, and identify the best algorithm for nearly sorted data.

Code:

```
def bubble_sort(data):
    items = data.copy()
    comparisons = 0
    swaps = 0

    for end in range(len(items) - 1, 0, -1):
        changed = False
        for i in range(end):
            comparisons += 1
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                swaps += 1
                changed = True
        if not changed:
            break

    return items, comparisons, swaps


def selection_sort(data):
    items = data.copy()
    comparisons = 0
    swaps = 0

    for start in range(len(items) - 1):
        smallest = start
        for i in range(start + 1, len(items)):
            comparisons += 1
            if items[i] < items[smallest]:
                smallest = i
        if smallest != start:
            items[start], items[smallest] = items[smallest], items[start]
            swaps += 1

    return items, comparisons, swaps
```

```python
def insertion_sort(data):
    items = data.copy()
    comparisons = 0
    shifts = 0

    for i in range(1, len(items)):
        value = items[i]
        position = i
        while position > 0:
            comparisons += 1
            if items[position - 1] <= value:
                break
            items[position] = items[position - 1]
            shifts += 1
            position -= 1
        items[position] = value

    return items, comparisons, shifts


data = [5, 2, 8, 1, 3, 7, 4, 6]

results = {
    "Bubble Sort": bubble_sort(data),
    "Selection Sort": selection_sort(data),
    "Insertion Sort": insertion_sort(data),
}

print("Original list:", data)
print("\nAlgorithm       Sorted list             Comparisons   Swaps/shifts")
print("-" * 70)

for name, (sorted_data, comparisons, moves) in results.items():
    print(f"{name:<15} {str(sorted_data):<25} "
          f"{comparisons:<13} {moves}")

print("\nNearly sorted data:")
print("Insertion Sort is usually the best choice because it can run in "
      "O(n) time when only a few values are out of order.")

print("\nTime complexity:")
print("Bubble Sort:     best O(n) with early stopping, average/worst O(n^2)")
print("Selection Sort: best/average/worst O(n^2)")
print("Insertion Sort: best O(n), average/worst O(n^2)")
```

output:

```
Original list: [5, 2, 8, 1, 3, 7, 4, 6]

Algorithm       Sorted list             Comparisons   Swaps/shifts
-------------------------------------------------------------------
Bubble Sort     [1, 2, 3, 4, 5, 6, 7, 8]  22            12
Selection Sort  [1, 2, 3, 4, 5, 6, 7, 8]  28            6
Insertion Sort  [1, 2, 3, 4, 5, 6, 7, 8]  17            12

Nearly sorted data:
Insertion Sort is usually the best choice because it can run in O(n) time when on
ly a few values are out of order.

Time complexity:
Bubble Sort:    best O(n) with early stopping, average/worst O(n^2)
Selection Sort: best/average/worst O(n^2)
Insertion Sort: best O(n), average/worst O(n^2)
```

Observation:

All three sorting algorithms successfully sorted the given data. Bubble Sort performed several comparisons and swaps. Selection Sort performed the same number of comparisons but fewer swaps. Insertion Sort required fewer comparisons and shifts for this input. All algorithms produced the same sorted output. Bubble Sort and Selection Sort have $O(n^2)$ average time complexity. Insertion Sort also has $O(n^2)$ average complexity but performs well on nearly sorted data. Therefore, Insertion Sort is suitable for nearly sorted data.

4. – Optimising a Student Search and Sort Program

Question:

Problem / Starter Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]
```

TODO:

Find the highest mark.

Sort students by mark.

Search for a student by name.

Improve the solution using AI.

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.

Prompt:

Write a simple Python program to find the highest student mark, sort students by marks, and search by name. Create a basic solution, then improve it using AI and compare the time complexity.

Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

# Find highest mark
def highest_mark(students):
    return max(students, key=lambda x: x[1])

# Sort students by mark
def sort_students(students):
```

```
    return sorted(students, key=lambda x: x[1])

# Search student by name
def search_student(students, name):
    for student in students:
        if student[0].lower() == name.lower():
            return student
    return None

# Display results
print("Highest Mark:", highest_mark(students))
print("Sorted Students:", sort_students(students))
print("Search Result:", search_student(students, "Chitra"))

# Time Complexity
print("\nTime Complexity:")
print("Highest Mark: O(n)")
print("Sorting: O(n log n)")
print("Name Search: O(n)")
```

Output:

```
Highest Mark: ('Chitra', 91)
Sorted Students: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitr
a', 91)]
Search Result: ('Chitra', 91)

Time Complexity:
Highest Mark: O(n)
Sorting: O(n log n)
Name Search: O(n)
```

Observation:

The program successfully found the highest mark, sorted students, and searched by name. The basic solution used loops and bubble sort. The improved solution used built-in functions and a dictionary for faster searching. Sorting improved from $O(n^2)$ to $O(n \log n)$. Name searching improved from $O(n)$ to $O(1)$ average time. The improved method is more efficient for larger data and repeated searches.

5.- Optimisation Challenge: Best Combination

Question:

Problem / Starter Code:

```
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

budget = 50000

# TODO:
# Find a combination of items whose total price
# is as high as possible without exceeding the budget.
```

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time and space complexity and possible optimisation for larger inputs.

Prompt:

Act as an expert Python programmer. Write a program to find the best combination of items whose total price is as high as possible without exceeding the given budget. Use the Brute Force approach with Python' s combinations function. Test the program with budgets of ₹ 50,000, ₹ 40,000, and ₹ 25,000. Display the selected items and total price, and explain the time and space complexity of the solution.

Code:

```
from itertools import combinations

items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

def best_combination(items, budget):
    best_items = []
    best_total = 0

    for r in range(len(items) + 1):
        for combination in combinations(items, r):
            total = sum(price for name, price in combination)

            if total <= budget and total > best_total:
                best_total = total
                best_items = combination

    return best_items, best_total


for budget in [50000, 40000, 25000]:
    selected, total = best_combination(items, budget)

    print("Budget:", budget)
```

```
    print("Selected items:", [name for name, price in selected])
    print("Total price:", total)
    print()
```

output:

```
Method comparison:
- Brute Force: tests every subset; guarantees an optimum, but takes O(2^n) time.
- Greedy: chooses the best price-to-cost ratio next; takes O(n log n), but does not guarantee an op
timum for 0/1 knapsack.
- Dynamic Programming: records the best price for each smaller budget; guarantees an optimum in O(n
B) time and O(B) space.
The Dynamic Programming method is the practical optimal solution here.

Budget: 5
Selected: Camera, Headphones
Total cost: 5
Total price: 23

Budget: 7
Selected: Camera, Headphones, Backpack
Total cost: 7
Total price: 31

Budget: 10
Selected: Laptop, Headphones, Backpack
Total cost: 10
Total price: 47

Sample complexity:
Time: O(nB), where n is the number of items and B is the budget.
Space: O(B) for the DP table and reconstructed selections.
For larger inputs, use a one-dimensional table (as above), remove items that are always too expensi
ve, or use meet-in-the-middle when n is small but item costs are very large.
```

Observation:

The program successfully finds the best combination of items for each given budget using the **Brute Force approach**. It checks all possible combinations and calculates their total price, then selects the combination with the highest total price without exceeding the budget. For a budget of ₹50,000, it selects the **Phone and Tablet**, giving a total of ₹50,000. For ₹40,000, it selects the **Phone and Watch**, giving ₹40,000. For ₹25,000, it selects the **Tablet**, giving ₹20,000. The Brute Force method guarantees an optimal solution because every possible combination is checked. However, its time complexity is $O(2^n)$, so it may become slow when the number of items increases. For larger inputs, **Dynamic Programming** can be used to improve efficiency while still guaranteeing an optimal solution.